Índice

Lo primero que llama la atención **2**
Gestión de Dependencias y Maven 2
Ignorando archivos de compilación (.gitignore) **3**
Sintaxis Básica **3**
Llaves `{}` en vez de indentación 3
Punto y coma `;` obligatorio 4
Variables y Tipos **5**
Java es fuertemente tipado 5
Inmutable vs Mutable 5
Visibilidad (`public` / `private`) **6**
Por qué existe este concepto 6
¿Qué pasa si no ponés modificador? 6
Niveles de visibilidad 7
Clases y Herencia **7**
Herencia con `extends` 7
Clases abstractas 7
Clases concretas (no abstractas) 8
Métodos **8**
`return` es obligatorio 8
`void` para métodos que no devuelven nada 9
Parámetros de métodos 9
Colecciones **9**
Java necesita tipo específico y `new` 9
Recorrer una colección 10
Otros tipos de colecciones 10
Comparación Directa Wollok → Java **10**
Override **11**
Main - Punto de Entrada **12**
Main vacío - No hace nada 12
Main que crea un Contenido y usa un método 12
Main que crea Contenido y Pelicula 12
Main que delega a otro método 12
¿Cuánto código debe tener el main? 13
Si tenés muchas acciones (como 50 tests) 13
¿Por qué existe el main? 14
¿Qué pasa si no tengo main? 14
Constructores **14**
Constructor en Java vs Wollok 14
¿Por qué `this`? 15
¿Qué pasa si no escribís constructor? 15
Constructor en clase abstracta 15
¿Por qué `protected` en vez de `private`? 16

¿Qué pasa si no escribís constructor y lo necesitás? 16

Esta guía explica los conceptos básicos de Java comparándolos con Wollok, para facilitar la traducción de un lenguaje a otro.

Lo primero que llama la atención

Lo primero que llama la atención es la estructura de directorios.

```
wtoj-plataforma-contenidos-CreadorSW-1
    ├── mvnw
    ├── mvnw.cmd
    ├── pom.xml
    ├── README.md
    ├── src
    │   ├── main
    │   │   └── java
    │   │       └── ar
    │   │           └── edu
    │   │               └── unahur
    │   │                   └── obj2
    │   │                       └── w2j
    │   │                           ├── contenidos
    │   │                           │   ├── Contenido.java
    │   │                           │   ├── Peliculas.java
    │   │                           │   └── series
    │   │                           │       ├── Episodio.java
    │   │                           │       ├── Serie.java
    │   │                           │       └── Temporada.java
    │   │                           └── Main.java
    │   └── test
    │       └── java
```

- Esa estructura es porque seguimos la convención de Maven.
- Cada clase en Java va en un archivo `.java` que se escribe con mayúscula.
- El `Main.java` viene con el repo porque se necesita como punto de entrada (esto se explica en la sección correspondiente).
- `mvnw` y `mvnw.cmd`: Scripts que permiten usar Maven sin instalarlo globalmente. El primero es para Linux/Mac, el segundo para Windows.
- `pom.xml`: Archivo de configuración de Maven que define las dependencias y cómo compilar el proyecto.

Gestión de Dependencias y Maven

Java utiliza herramientas como **Maven** para gestionar las librerías externas. Los editores modernos (como Zed) detectan automáticamente la configuración del proyecto:

- **Detección**: El servidor de lenguaje (JDT.LS) busca el archivo `pom.xml`. Si existe, configura el proyecto siguiendo el estándar de Maven.

- **Maven Wrapper (`mvnw`)**: Permite ejecutar Maven sin tenerlo instalado globalmente, asegurando que todos los desarrolladores usen la misma versión.
- **Repositorio Local**: Las dependencias se descargan una sola vez a la carpeta `~/.m2/repository`.
  - La primera compilación es lenta porque descarga todo de internet.
  - Las siguientes son rápidas porque Maven reutiliza lo que ya tiene en disco.
  - Esto permite trabajar offline una vez que las dependencias iniciales fueron obtenidas.

Ignorando archivos de compilación (.gitignore)

Cuando subís el proyecto a Git, **no** deben incluirse los archivos generados automáticamente:

- `target/`: Directorio donde Maven guarda los `.class` compilados y los resultados de los tests. Puede ocupar varios megabytes.
- `*.class`: Archivos de bytecode Java, por si compilás un archivo suelto a mano fuera de Maven.

Estos archivos **no** deben subirse porque:

- Son generados automáticamente desde cualquier código fuente.
- Ocupan espacio innecesario.
- Cada desarrollador los genera localmente al compilar.

Todo lo demás **debe quedarse** en el repositorio:

- `.mvn/`, `mvnw`, `mvnw.cmd`, `pom.xml`, `src/`

Esto se configura en un archivo `.gitignore` en la raíz del proyecto.

Sintaxis Básica

Llaves {} en vez de indentación

En Java, las llaves `{}` definen los bloques de código (clases, métodos, condiciones). La indentación es solo visual, no sintáctica.

Java:

```java
public class Contenido {
    private String titulo;
    private int costoBase;

    public int costo() {
        return this.costoBase;
    }
}
```

Wollok:

```wollok
class Contenido {
    const titulo
    var costoBase

    method costo() = costoBase
}
```

¿Qué pasa si no usás las llaves?

Java:

```java
public class Contenido {
    private String titulo  // ERROR: missing } at end of class
```

El compilador tira error de sintaxis porque no puede determinar dónde termina el bloque.

Punto y coma ; obligatorio

Java requiere ; al final de cada instrucción. Es obligatorio porque una línea puede contener varias instrucciones.

Java:

```java
int x = 5;
x = x + 1;
```

Wollok:

```wollok
var x = 5
x = x + 1
```

¿Qué pasa si no ponés ;?

Java:

```java
int x = 5  // ERROR: syntax error, insert ';' to complete statement
x = x + 1;
```

El compilador informa exactamente dónde falta el ;.

Variables y Tipos

Java es fuertemente tipado

En Java **siempre** debés indicar el tipo de la variable. No podés dejar que el compilador lo infiera.

Java:

```java
String titulo = "Recuerdos";
int costoBase = 20;
boolean esPopular = true;
double promedio = 22.5;
```

Wollok:

```wollok
const titulo = "Recuerdos"
var costoBase = 20
var esPopular = true
var promedio = 22.5
```

¿Qué pasa si usás un tipo incorrecto?

Java:

```java
String titulo = 123;  // ERROR: incompatible types: int cannot be converted to String
```

El compilador detecta el error antes de ejecutar el programa.

Inmutable vs Mutable

Java:

```java
final String titulo = "Recuerdos";  // no se puede cambiar (equivalente a const)
int costoBase = 20;                 // sí se puede cambiar (equivalente a var)
```

Wollok:

```wollok
const titulo = "Recuerdos"   // inmutable
var costoBase = 20            // mutable
```

¿Qué pasa si intentás modificar una variable `final`?

Java:

```java
final String titulo = "Recuerdos";
titulo = "Otro título";  // ERROR: cannot assign a value to final variable titulo
```

El compilador lanza error de compilación.

Visibilidad (`public` / `private`)

Por qué existe este concepto

El **encapsulamiento** protege los datos de modificaciones accidentales o incorrectas. Cada clase decide qué propiedades y métodos son accesibles desde afuera.

Java:

```java
public class Contenido {
    private String titulo;  // solo la clase Contenido lo ve

    public String getTitulo() {  // acceso controlado
        return this.titulo;
    }

    public void setTitulo(String nuevoTitulo) {  // validación antes de modificar
        this.titulo = nuevoTitulo;
    }
}
```

Wollok:

```wollok
class Contenido {
    const titulo
    var costoBase
}
```

En Wollok no existe encapsulamiento: cualquiera puede ver y modificar cualquier atributo.

¿Qué pasa si no ponés modificador?

Java:

```java
class Contenido {
    String titulo;  // package-private (default)
}
```

Cualquier clase en el **mismo paquete** puede ver y modificar `titulo`. Es una vulnerabilidad porque se rompe el control sobre cómo se accede a los datos.

Niveles de visibilidad

Modificador	Alcance
`public`	Cualquier clase desde cualquier paquete
`private`	Solo dentro de la misma clase
(ninguno)	Package-private (solo clases del mismo paquete)

Clases y Herencia

Herencia con `extends`

Java:

```java
public class Pelicula extends Contenido {
    // hereda titulo y costoBase automáticamente
}
```

Wollok:

```wollok
class Pelicula inherits Contenido {
    // vacía, hereda todo
}
```

¿Qué pasa si heredás de una clase que no existe?

Java:

```java
public class Pelicula extends Contenido { }
// ERROR: Contenido cannot be resolved to a type
```

El compilador no encuentra la clase padre.

Clases abstractas

Una clase abstracta no se puede instanciar directamente. Sirve como base para otras clases.

Java:

```java
public abstract class Contenido {
    public abstract int costo();  // las subclases DEBEN implementar
}
```

Wollok:

```wollok
class Contenido {
    method costo()  // sin cuerpo = abstracto
}
```

¿Qué pasa si no implementás el método abstracto?

Java:

```java
public class Pelicula extends Contenido {
    // ERROR: The type Pelicula must implement the inherited abstract method Contenido.costo()
}
```

El compilador obliga a implementar todos los métodos abstractos.

Clases concretas (no abstractas)

Java:

```java
public class Pelicula extends Contenido {
    @Override
    public int costo() {
        return this.getCostoBase();
    }
}
```

Wollok:

```wollok
class Pelicula inherits Contenido {
    // vacía, hereda el costo() de Contenido si existiera
}
```

Métodos

`return` es obligatorio

Si declarás un método que devuelve algo, **debes** retornar un valor.

Java:

```java
public int costo() {
    return this.costoBase;
}
```

Wollok:

```wollok
method costo() = costoBase
```

¿Qué pasa si no ponés `return`?

Java:

```java
public int costo() {
    costoBase;  // ERROR: missing return statement
}
```

El compilador detecta que el método promete devolver `int` pero no lo hace.

`void` para métodos que no devuelven nada

Java:

```java
public void agregarTemporada(Temporada t) {
    temporadas.add(t);
}
```

Wollok:

```wollok
method agregarTemporada(unaTemporada) {
    temporadas.add(unaTemporada)
}
```

¿Qué pasa si intentás usar `return` en un método `void`?

Java:

```java
public void agregarTemporada(Temporada t) {
    return true;  // ERROR: void methods cannot return a value
}
```

Parámetros de métodos

Java:

```java
public void agregarTemporada(Temporada unaTemporada) {
    temporadas.add(unaTemporada);
}
```

Wollok:

```wollok
method agregarTemporada(unaTemporada) {
    temporadas.add(unaTemporada)
}
```

Colecciones

Java necesita tipo específico y `new`

Java:

```java
List<Temporada> temporadas = new ArrayList<>();
temporadas.add(unaTemporada);
temporadas.size();
```

Wollok:

```wollok
const temporadas = []
temporadas.add(unaTemporada)
temporadas.size()
```

Recorrer una colección

Java:

```java
int suma = temporadas.stream().mapToInt(t -> t.costo()).sum();
```

Wollok:

```wollok
var suma = temporadas.sum({ t => t.costo() })
```

Otros tipos de colecciones

Java:

```java
Set<String> titulos = new HashSet<>();
Map<String, Integer> duracion = new HashMap<>();
```

Wollok:

```wollok
// Wollok no distingue entre Set, Map, List
// usa simplemente listas
const titulos = []
const duracion = []
```

Comparación Directa Wollok → Java

Concepto	Wollok	Java
Inmutable	**`const`**	`final`
Mutable	**`var`**	`tipo` (sin final)
Herencia	**`class X inherits Y`**	`class X extends Y`
Clase abstracta	**método sin cuerpo**	`abstract class X`

Concepto	Wollok	Java
Method abstracto	**`method costo()`**	`public abstract int costo()`
Visibilidad	**no existe**	`public` / `private`
Lista	**`[]`**	`new ArrayList<>()`
Tamaño	**`.size()`**	`.size()`
Sumar elementos	**`.sum({x => x.costo()})`**	`.stream().mapToInt(x -> x.costo()).sum()`
Punto y coma	**opcional**	obligatorio
Llaves	**no se usan**	obligatorias
Constructor	**automático**	hay que escribirlo

Override

En Java es **obligatorio** usar la anotación `@Override` cuando sobrescribís un método de la clase padre.

Java:

```java
@Override
public int costo() {
    return 3;
}
```

Wollok:

```wollok
override method costo() {
    return 3
}
```

¿Qué pasa si te olvidás `@Override`?

Java:

```java
public int costo() {  // se supone que es override pero sin anotación
```

```
    return 3;
}
```

Si no es realmente un override (por ejemplo, si el método padre tiene otra firma), Java **no te avisa**. Podés tener un método completamente nuevo sin querer y el compilador no dice nada.

Main - Punto de Entrada

Todo programa Java necesita un método `main` como punto de entrada. Sin él, Java no sabe qué ejecutar primero.

```java
public static void main(String[] args) {
    // Acá empieza todo
}
```

Main vacío - No hace nada

```java
public static void main(String[] args) {
    // vacío
}
```

El programa compila, corre, pero no pasa nada visible.

Main que crea un Contenido y usa un método

```java
public static void main(String[] args) {
    Contenido pelicula = new Contenido("Recuerdos", 20);
    System.out.println(pelicula.costo());  // imprime 20
}
```

Resultado: `20`

Main que crea Contenido y Pelicula

```java
public static void main(String[] args) {
    Contenido pelicula = new Contenido("Recuerdos", 20);
    Contenido otraPelicula = new Pelicula("Avatar", 30);

    System.out.println(pelicula.costo());       // 20
    System.out.println(otraPelicula.costo());    // 30
}
```

Main que delega a otro método

El main nunca tiene toda la lógica. Solo crea objetos y delega.

```java
public static void main(String[] args) {
```

```java
    Plataforma plataforma = new Plataforma();
    plataforma.iniciar();  // toda la lógica está en iniciar(), no acá
}
```

¿Cuánto código debe tener el main?

Mínimo posible. Solo crea objetos y delega. No pongas toda la lógica en el main.

Mal:

```java
public static void main(String[] args) {
    // 200 líneas de código todo mezclado
    // crear objetos
    // llamar métodos
    // hacer cálculos
    // imprimir resultados
}
```

Bien:

```java
public static void main(String[] args) {
    Plataforma plataforma = new Plataforma();
    plataforma.iniciar();
}
```

Si tenés muchas acciones (como 50 tests)

Tenés tres opciones:

Opción 1: Framework de testing (JUnit) No ponés nada en el main. Los tests son clases separadas con `@Test`. El framework los ejecuta.

Opción 2: Clase Ejecutor separada

```java
// Main.java
public class Main {
    public static void main(String[] args) {
        Ejecutor.ejecutarTodo();
    }
}

// Ejecutor.java
public class Ejecutor {
    public static void ejecutarTodo() {
        Contenido pelicula = new Contenido("Recuerdos", 20);
        System.out.println(pelicula.costo());
    }
```

```
}
```

Opción 3: Métodos estáticos llamados desde main

```java
public static void main(String[] args) {
    testCostoPelicula();
    testCostoSerie();
}

public static void testCostoPelicula() {
    Contenido pelicula = new Contenido("Recuerdos", 20);
    System.out.println(pelicula.costo());
}

public static void testCostoSerie() {
    // ...
}
```

¿Por qué existe el main?

Java no tiene un REPL como Wollok. Necesita saber **exactamente** dónde empieza. Sin `main`, no hay punto de partida.

¿Qué pasa si no tengo main?

```
Error: Main method not found in class Contenido, please define the main method as:
public static void main(String[] args)
```

Constructores

En Java, si necesitás inicializar objetos con parámetros, debés escribir el constructor explícitamente. Wollok lo hace automáticamente, Java no.

Constructor en Java vs Wollok

Java:

```java
public class Contenido {
    private final String titulo;
    private double costoBase;

    public Contenido(String titulo, double costoBase) {
        this.titulo = titulo;
        this.costoBase = costoBase;
    }
```

```
}
```

Wollok:

```wollok
class Contenido {
    const titulo
    var costoBase
}
// Wollok genera automáticamente el constructor con los parámetros
```

¿Por qué `this`?

`this.titulo` significa «el campo titulo de este objeto». Sin `this`, Java no sabe si `titulo` se refiere al parámetro o al campo.

```Java
public Contenido(String titulo, double costoBase) {
    this.titulo = titulo;          // this.titulo = campo, titulo = parámetro
    this.costoBase = costoBase;
}
```

¿Qué pasa si no escribís constructor?

```Java
public class Contenido {
    private String titulo;
    private double costoBase;
}
// Java crea automáticamente: Contenido() {}
```

Si no definís ningún constructor, Java crea uno automático sin parámetros. Pero si definís **al menos uno con parámetros**, el automático desaparece.

Constructor en clase abstracta

```Java
public abstract class Contenido {
    private final String titulo;
    protected double costoBase;  // protected para que las subclases accedan

    public Contenido(String titulo, double costoBase) {
        this.titulo = titulo;
        this.costoBase = costoBase;
    }
}
```

¿Por qué `protected` en vez de `private`?

Modificador	Quién puede verlo
`private`	**Solo la clase misma**
`protected`	**La clase y sus subclases**

Si `costoBase` fuera `private`, `Pelicula` (que hereda de `Contenido`) no podría acceder a `costoBase`.

Wollok: No existe este concepto. Todos los atributos son accesibles.

¿Qué pasa si no escribís constructor y lo necesitás?

```java
Contenido c = new Contenido();  // ERROR: no default constructor exists
```

Si definiste un constructor con parámetros y necesitás el sin parámetros, debés escribirlo vos.